Week 4 – Design Studio: Answers

Design the delivery-time estimate

> Read this after the studio. It isn't the only good answer; it's what a strong answer contains. The system is a scorer with both placements: a nightly job that computes what can be known before any order, and an online path that computes what can only be known at checkout. The interesting part is the data: where the truth comes from, when, and which orders never produce one.

The numbers, as given

The prompt supplies them; what a strong answer adds is the implication of each.

NUMBER	VALUE	IMPLICATION
estimates per second, Friday 19:00	about 2,000 orders a minute at the peak; but the list page shows 20 restaurants, so **about 700 estimates a second**, bursty	the model cannot be a network call per estimate; it is an inference inside the web server's process
latency, one estimate	**under 200 ms** for the page, so under 5 ms for each of 20 estimates; nothing in that path may wait on a database round trip per restaurant	the restaurant and zone parts are precomputed and read in one lookup
cost per month	the nightly job: pennies; the online path: free, it is arithmetic inside a process already running	the budget is engineering time, not inference

Any design that calls a model service per restaurant at the peak fails the latency number before it is drawn.

1. How it is built

The two placements

Before the order, nightly. For every restaurant: preparation time at the median and the 90th percentile, by hour of day and day of week, from the last four weeks of delivered orders. For every delivery zone: typical travel time to each neighbouring zone, courier supply by hour. These change slowly and depend only on history, so a scheduler computes them once a night and writes a **feature table** keyed by restaurant id and zone id, with an `as_of` time on every row.

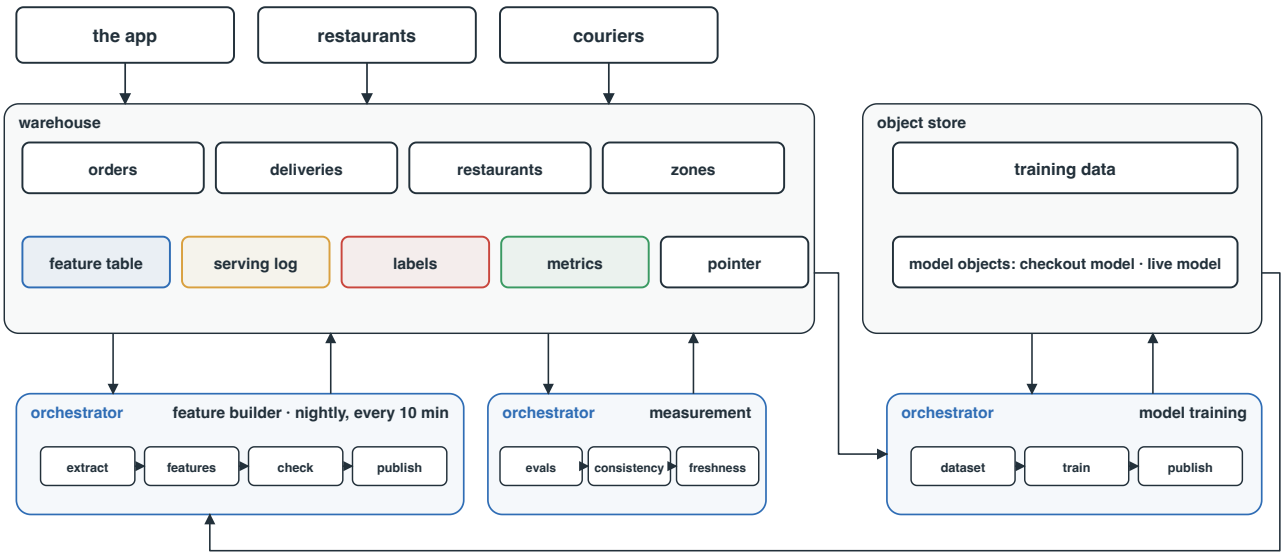
Before the order, every ten minutes. The live parts: how many couriers are free in the zone right now, how many orders each restaurant has open. Same table, a different schedule, a different as-of. These are the rows that go stale first and matter most at the Friday peak.

At the order. This basket (item count, a dish the restaurant marks as slow), this address (distance from the restaurant, computed from two coordinates), this moment (minute of the day). Joined with the table rows for this restaurant and this zone; one feature vector; one inference.

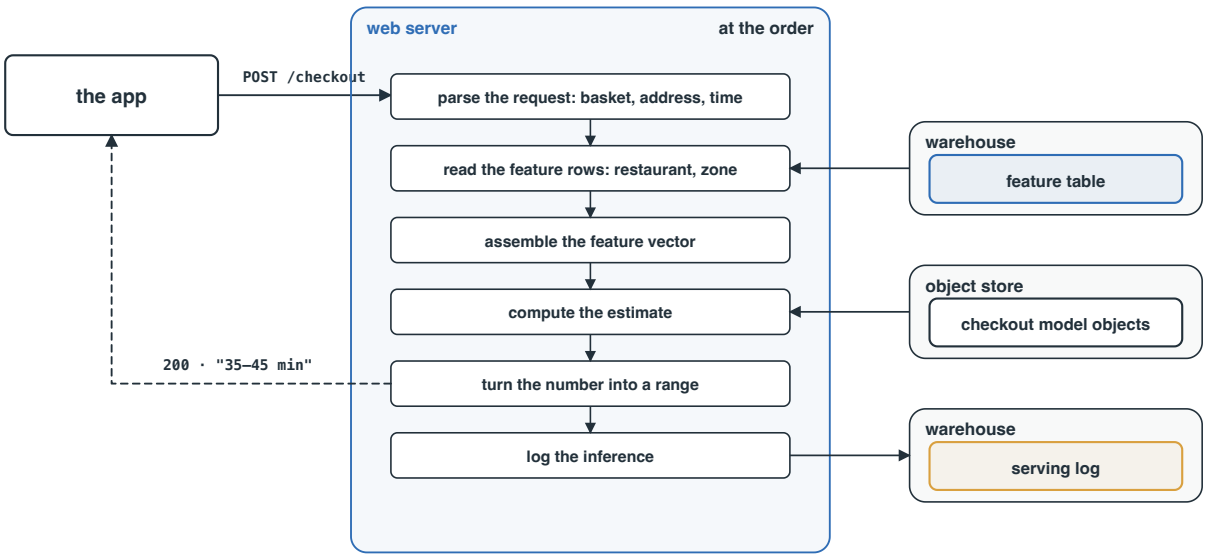
The table between. The feature table is the seam. The online path reads it with one lookup per restaurant and reports the age of what it read; a row older than an hour for the live features is flagged on the estimate, and a row older than a day for the nightly ones is a failed job someone should know about.

The architecture

The orchestrated parts: what runs before any order, on a schedule.



The web server: what runs at the order, inside one request.



The path of one estimate:

1. The app requests the restaurant list, or the checkout page. One request, up to twenty estimates.
2. Feature assembly, inside the web server's process: the basket and address from the request, the restaurant and zone rows from the feature table, read by id. One lookup, a few milliseconds.
3. The model, in process: the model objects loaded when the server started. One inference per estimate, in memory, whatever the model type. The range shown is a rule around the model's number: the median plus and minus a margin that depends on the zone's variance.
4. The page returns in under 200 ms. The estimate was never a network call.

Then, off the request: the **serving log** writes one row per estimate shown at checkout: order id, the feature vector exactly as assembled, the number produced, the model version, the as-of of every table row used, the time. The list-page estimates are sampled, not all logged, at 700 a second.

Tables. `restaurants` and `zones` (reference data), `feature_table` (written by the scheduler, read by the request path, as-of on every row), `orders` (written at checkout), `serving_log` (written off the request), `deliveries` (written by the courier app when the order is marked delivered), `training_set` (built from the join of `serving_log` and `deliveries`, never from `orders` alone).

Sync and async. The request path is synchronous end to end and contains no model call that leaves the process. Everything else is asynchronous: the nightly and ten-minute jobs, the serving log write, the dispatch, the delivery, the label join, the retrain.

2. How its data is handled

Where the label comes from. The courier marks the order delivered. The actual minutes from checkout to that tap is the label. It arrives 30 to 60 minutes after the estimate was shown.

Which orders never get one. Cancelled orders. And, worse, cancelled orders are not random: a long estimate makes a customer cancel, so the orders that never produce a label are disproportionately the ones the model predicted would be slow. Next month's training set over-represents the orders the estimate did not scare away. Left alone, the model learns that long estimates do not happen, and shortens them, which is the one thing it must not do.

The remedy is the same one as for any score that selects its own labels: a small random slice of orders is shown a conservative, model-free estimate (the zone's historical median), so the system keeps observing actual delivery times on orders the model did not shape. Its cost is a slightly worse estimate on one order in two hundred; its value is that the training set has a slice the model did not select.

What the training table knew that checkout cannot. The courier assigned, the courier's distance from the restaurant when assigned, the restaurant's actual preparation time for this order, the time the restaurant accepted it. All known for every past order; none known at checkout. A training table built from `orders` joined with `deliveries` has all of them, produces a model that looks superb offline, and gives production a model whose best features are empty. The rule: **a training row is the serving-log row**, the feature vector exactly as it was assembled when the estimate was shown, joined with the label. Nothing else enters. If a feature was not in the serving log, it was not available at serving time, and it does not exist for training.

What is written down at each estimate. The serving-log row: the order id, the feature vector as assembled, the number produced, the model version, the as-of of every table row used, the time. Without it, a complaint about one estimate cannot be investigated and the order cannot become a training row.

3. How it is measured

The promise. The estimate is a range; the requirement is a rate: the share of orders delivered inside the range shown, per slice (restaurant, zone, hour, basket size, and the Friday peak on its own), with an interval around it. The serving log makes this measurable for every order that was delivered; the random slice makes it honest, because it contains orders the estimate did not shape.

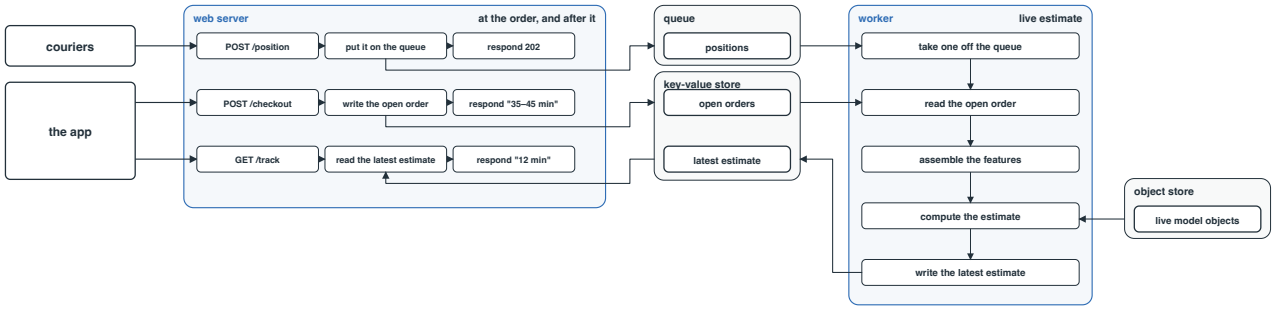
The system. The page's estimate at the 99th percentile under 200 ms, measured at the page. Served equals recomputed: an estimate recomputed from its logged feature vector must match exactly. The pipeline: age of the nightly rows and of the ten-minute rows when read, share of estimates with a serving-log row, share of delivered orders that got a label joined within the hour.

4. The live estimate *(optional)*

What changes. No request asks for the number; the courier's phone reports a position every few seconds, and each report is a reason to re-estimate. The model runs per report, not per page load. One order now produces many estimates and still only one label.

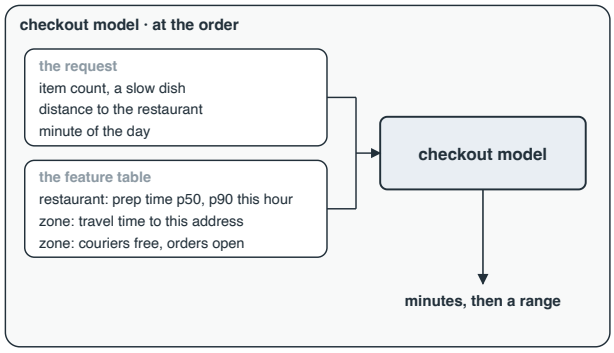
What triggers a re-estimate. A position report that moves the courier more than a block; the restaurant marking the order ready; the courier picking it up; and silence. A courier who has not reported for two minutes is information, and a re-estimate with that fact in the features is the honest one.

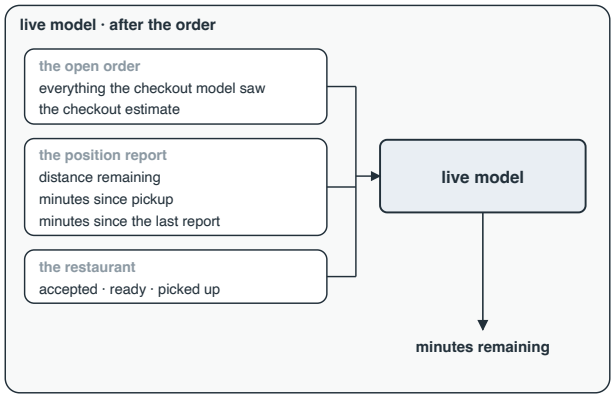
What is built.



Three things are new, and all three are things we already have. The courier's phone sends `POST /position` to the web server, which puts the report on a **queue** and returns 202; the phone never waits on a model. A **worker** takes reports off the queue one at a time: it reads the order from a **key-value store**, where the web server wrote the features as served at checkout, assembles them with the new position, runs the model, and writes the latest estimate back to the key-value store by order id.

A second model. The live estimate is not the checkout model run again. Its inputs include the courier's position, the minutes elapsed and the restaurant's state, which the checkout model never sees, and it is trained from the logged revisions joined with the labels, not from checkout rows. So there are two model objects in the object store: the checkout model, loaded by the web server, and the live model, loaded by the worker. The training job produces both, each from its own serving rows, and the measurement scores each against the same actual delivery time.





The app asks `GET /track` every few seconds, and the web server answers with that one key. The queue is the one from the support desk: on a Friday night it absorbs the burst of reports, and the worker drains it at the model's pace; the tracking page shows the newest estimate the worker reached, with its age.

What is written down at each estimate. The same shape as the serving-log row: the order id, the report that triggered the revision, the features as assembled (position, minutes elapsed, the restaurant's state), the number produced, the model version, the time. The worker appends it to the serving log as well as writing the key, so the label join and the measurement see every revision. Without the triggering report, a complaint about a jump from 12 to 25 minutes cannot be explained.

Which estimate the label scores. All of them, each against the same actual delivery time, but reported separately. The checkout promise is the one the customer decided on, and it keeps its own rate. The live estimates are scored by how far out they were made: an estimate two minutes before delivery should be nearly exact, one thirty minutes out should not, and a single number across them hides both. Then a measure the checkout estimate did not need: **revisions**. How often the number shown moved by more than five minutes, and in which direction. A promise kept by moving it is a different promise, and the customer experiences the moves.